

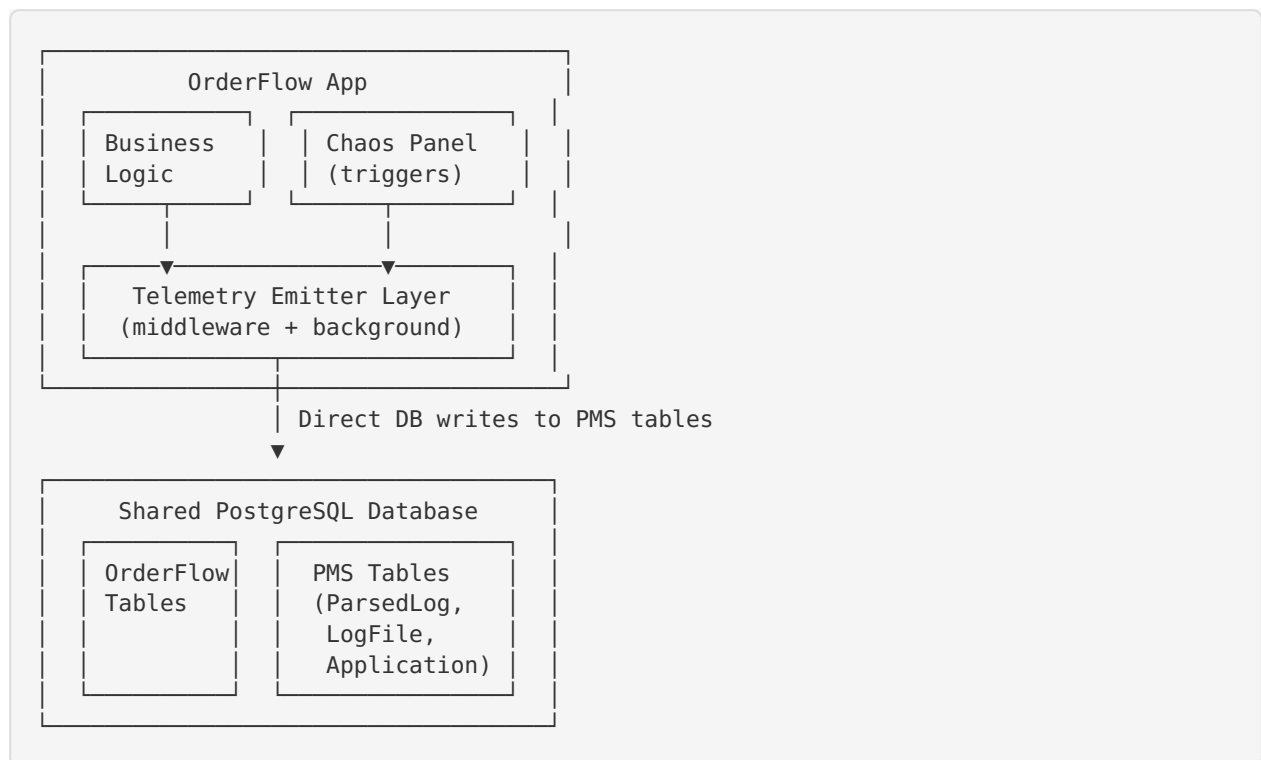
OrderFlow — PMS Companion Test Application

Purpose

OrderFlow is a realistic order-processing web application that shares the PMS PostgreSQL database. Its primary purpose is to **generate real telemetry** (HTTP logs, database activity, job events, errors, metrics) that the PMS system can ingest, analyze, and predict against.

It also includes a **Chaos Engineering Panel** for triggering failure scenarios on demand.

Architecture



CRITICAL: Unplug Mechanism

The OrderFlow app MUST be production-safe. ALL telemetry writes to PMS tables are gated behind:

```
# .env
PMS_TELEMETRY_ENABLED=true      # Set to "false" to disable all PMS writes
PMS_APPLICATION_ID=<id>        # The Application.id in PMS (created during setup)
```

Implementation Rules:

1. **Single gateway:** ALL PMS writes go through a `lib/pms-telemetry.ts` service. No direct Prisma calls to PMS tables anywhere else.
2. **Kill switch check:** `pmsTelemetry.emit(...)` checks `PMS_TELEMETRY_ENABLED !== 'false'` before every write. When disabled, it silently returns `{ skipped: true }`.
3. **Chaos panel disabled:** When telemetry is off, the Chaos Panel UI shows a banner: “Telemetry disabled — chaos scenarios unavailable” and all trigger buttons are disabled.
4. **No PMS table migrations:** OrderFlow MUST NOT run `prisma db push` or `prisma migrate` against PMS tables. It only READS the PMS schema definition and WRITES rows. Schema ownership stays with PMS.
5. **Graceful degradation:** If `PMS_APPLICATION_ID` is missing or the Application row doesn't exist, telemetry silently disables itself (logs a warning, doesn't crash).

Shared Database — PMS Tables (READ/WRITE)

OrderFlow writes to these existing PMS tables. **Do not modify their schema.**

Application (READ-ONLY)

OrderFlow reads its own Application row for metadata. The Application is pre-created by PMS.

LogFile (WRITE)

```
model LogFile {
  id          String      @id @default(cuid())
  fileName    String      // e.g. "orderflow/http/2026-04-27T10:00:00"
  fileType    String      // "orderflow-telemetry"
  logSource    String      // e.g. "orderflow.http", "orderflow.database", etc.
  fileSize    Int          // approximate byte size of the batch
  cloudStoragePath String  // "orderflow/<source>/<timestamp>"
  isPublic     Boolean     @default(false)
  status       String      // always "processed"
  recordCount  Int?        // number of entries
  uploadedBy   String      // FK to User.id (use the admin user)
  applicationId String?    // FK to Application.id = PMS_APPLICATION_ID
  dataSourceId String?    // null (OrderFlow isn't a DataSource)
  ingestionFeed String?    // the telemetry source type
}
```

ParsedLog (WRITE)

```
model ParsedLog {
  id          String      @id @default(cuid())
  logFileId   String      // FK to the LogFile created above
  timestamp    DateTime    // when the event occurred
  logLevel    String      // "info", "warning", "error", "critical"
  source       String      // e.g. "orderflow.http", "orderflow.database"
  message      String      // human-readable event description
  rawData     String?     // optional JSON of raw event data
  features     String?     // optional JSON of extracted numeric features
}
```

Telemetry Source Types

OrderFlow emits telemetry under these `source` / `logSource` values:

Source	logSource	Description
<code>orderflow.http</code>	<code>orderflow.http</code>	HTTP request/response logs
<code>orderflow.database</code>	<code>orderflow.database</code>	Query execution, slow queries, connection pool
<code>orderflow.jobs</code>	<code>orderflow.jobs</code>	Background job execution events
<code>orderflow.errors</code>	<code>orderflow.errors</code>	Unhandled exceptions, validation failures
<code>orderflow.metrics</code>	<code>orderflow.metrics</code>	Periodic health snapshots (CPU, memory, connections)
<code>orderflow.chaos</code>	<code>orderflow.chaos</code>	Chaos experiment start/stop/status events

Feature Fields (JSON in `features` column)

Each source type emits specific numeric features for ML consumption:

orderflow.http:

```
{ "responseTimeMs": 142, "statusCode": 200, "requestSizeBytes": 1024, "responseSize-Bytes": 4096 }
```

orderflow.database:

```
{ "queryDurationMs": 85, "rowsAffected": 12, "connectionPoolActive": 8, "connection-PoolIdle": 2 }
```

orderflow.jobs:

```
{ "executionDurationMs": 3400, "retryCount": 0, "queueDepth": 5 }
```

orderflow.metrics:

```
{ "cpuUsagePct": 45, "memoryUsageMb": 512, "heapUsedMb": 320, "activeConnections": 8, "requestsPerMinute": 120, "errorRate": 0.02 }
```

OrderFlow Business Tables (NEW — owned by OrderFlow)

These tables are created and managed by OrderFlow. Use a `of` prefix (OrderFlow) to avoid collisions.

```

model OfProduct {
  id          String    @id @default(cuid())
  name        String
  sku         String    @unique
  description String?
  price       Float
  stock       Int       @default(0)
  category    String
  isActive    Boolean   @default(true)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  orderItems  OfOrderItem[]
}

model OfCustomer {
  id          String    @id @default(cuid())
  email       String    @unique
  name        String
  phone       String?
  tier         String    @default("standard") // standard, premium, enterprise
  createdAt   DateTime  @default(now())
  orders      OfOrder[]
}

model OfOrder {
  id          String    @id @default(cuid())
  orderNumber String    @unique // OF-0001, OF-0002, ...
  customerId  String
  customer    OfCustomer @relation(fields: [customerId], references: [id])
  status      OfOrderStatus @default(PENDING)
  totalAmount Float
  notes       String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  items       OfOrderItem[]
  jobs        OfJob[]
}

enum OfOrderStatus {
  PENDING
  VALIDATED
  PROCESSING
  FULFILLED
  COMPLETED
  CANCELLED
  FAILED
}

model OfOrderItem {
  id          String    @id @default(cuid())
  orderId     String
  order       OfOrder   @relation(fields: [orderId], references: [id], onDelete: Cascade)
  productId   String
  product     OfProduct @relation(fields: [productId], references: [id])
  quantity    Int
  unitPrice   Float
}

model OfJob {
  id          String    @id @default(cuid())
  type        String    // "inventory_sync", "report_generation", "or-

```

```

der_fulfillment", "cleanup"
    status      String      @default("queued") // queued, running, completed, failed, cancelled
    orderId     String?
    order       OfOrder?    @relation(fields: [orderId], references: [id], onDelete: SetNull)
    payload     String?      @db.Text // JSON
    result      String?      @db.Text // JSON
    retryCount  Int          @default(0)
    maxRetries  Int          @default(3)
    startedAt   DateTime?
    completedAt DateTime?
    error       String?
    createdAt   DateTime     @default(now())
}

// Tracks active chaos experiments
model OfChaosExperiment {
    id          String      @id @default(cuid())
    scenario    String      // "slow_query", "error_burst", "memory_pressure", etc.
    status      String      @default("active") // active, completed, aborted
    intensity   Int         @default(50) // 0-100
    durationSec Int         @default(300) // target duration
    config      String?     @db.Text // JSON scenario-specific config
    startedAt   DateTime    @default(now())
    endedAt     DateTime?
    metrics     String?     @db.Text // JSON results/metrics from the experiment
}

```

Pages & UI

1. Dashboard (/)

- Order stats: total, by status, revenue
- Recent orders table
- Active jobs count
- System health indicators (from telemetry)
- Link to Chaos Panel

2. Products (/products)

- Product grid/table with CRUD
- Stock levels with low-stock warnings
- Bulk operations (generates DB load)

3. Orders (/orders)

- Order list with status pipeline visualization
- Create order form (selects customer + products)
- Order detail with status progression
- Each status transition generates telemetry

4. Customers (/customers)

- Customer list/table
- Basic CRUD

5. Jobs (/jobs)

- Background job queue viewer
- Job types: inventory sync, report gen, order fulfillment, cleanup
- Manual trigger buttons
- Status indicators

6. Chaos Panel (/chaos) 🔥

- Grid of chaos experiment cards
- Each card: description, intensity slider (0-100), duration picker, Start/Stop button
- Active experiments banner at top
- Live status indicators
- Results/metrics after experiment completes

Chaos Scenarios:

Scenario	Key	What It Does
Slow Queries	<code>slow_query</code>	Adds artificial delays to product/order queries
Error Burst	<code>error_burst</code>	Triggers rapid HTTP 500 errors on order endpoints
Memory Pressure	<code>memory_pressure</code>	Allocates growing arrays, reports rising memory in metrics
Connection Exhaustion	<code>connection_exhaustion</code>	Opens many DB connections, holds them
Cascading Failure	<code>cascade_failure</code>	Order processing fails → job fails → cleanup fails
Gradual Degradation	<code>gradual_degradation</code>	Slowly increases response times over duration
Job Queue Backup	<code>job_queue_backup</code>	Pauses job processing, queue grows
Intermittent Flapping	<code>intermittent_flap</code>	Alternates healthy/error every 2 minutes

7. Settings (/settings)

- PMS Telemetry toggle (reads/writes `PMS_TELEMETRY_ENABLED`)
 - Connection status to PMS
 - Application ID display
-

Telemetry Emitter Layer (lib/pms-telemetry.ts)

Core Service

```
class PmsTelemetryService {
  private enabled: boolean;
  private applicationId: string | null;
  private adminUserId: string | null;

  // Check enabled + resolve IDs on first call
  async initialize(): Promise<void>;

  // Gate check – returns false if disabled
  isEnabled(): boolean;

  // Write a batch of events to PMS LogFile + ParsedLog
  async emit(source: string, entries: TelemetryEntry[]): Promise<EmitResult>;

  // Convenience: emit a single event
  async emitOne(source: string, entry: TelemetryEntry): Promise<EmitResult>;
}

interface TelemetryEntry {
  timestamp: Date;
  logLevel: "info" | "warning" | "error" | "critical";
  message: string;
  rawData?: Record<string, unknown>;
  features?: Record<string, number>;
}

interface EmitResult {
  ok: boolean;
  skipped?: boolean; // true when telemetry disabled
  rowsEmitted?: number;
}
```

Middleware (middleware.ts or API wrapper)

- Intercepts every API response
- Emits `orderflow.http` entry: method, path, status, duration, user-agent
- Batches entries (flush every 10 entries or 5 seconds)

Background Health Emitter

- Runs on a 60-second interval (via `setInterval` in a server-side singleton)
- Collects: `process.memoryUsage()` , active DB connections, request count, error count
- Emits `orderflow.metrics` batch

Database Query Logger

- Prisma middleware (`$use`) that logs query durations
- Emits `orderflow.database` entries for queries > 100ms
- All queries logged when chaos `slow_query` is active

Job Logger

- Every job start/complete/fail emits `orderflow.jobs` entry

Seed Data

- **50 products** across 5 categories (Electronics, Clothing, Food, Office, Tools)
 - **10 customers** with varied tiers
 - **20 initial orders** in various statuses
 - **5 background jobs** in various states
-

API Routes

Business APIs

- GET/POST /api/products — list/create products
- GET/PATCH/DELETE /api/products/[id] — product CRUD
- GET/POST /api/orders — list/create orders
- GET/PATCH /api/orders/[id] — order detail/update status
- POST /api/orders/[id]/advance — advance order to next status
- GET/POST /api/customers — list/create customers
- GET/POST /api/jobs — list/trigger jobs
- POST /api/jobs/[id]/run — execute a job

Chaos APIs

- GET /api/chaos — list active experiments
- POST /api/chaos/start — start a chaos experiment
- POST /api/chaos/stop — stop an experiment
- GET /api/chaos/scenarios — available scenarios with descriptions

Telemetry APIs

- GET /api/telemetry/status — is telemetry enabled? connected to PMS?
 - POST /api/telemetry/toggle — enable/disable telemetry
 - GET /api/telemetry/stats — emission counts, last flush time
-

Environment Variables

```
# Database (shared with PMS – auto-configured by Abacus AI)
DATABASE_URL=postgresql://...

# PMS Integration
PMS_TELEMETRY_ENABLED=true           # "false" to disable all PMS writes
PMS_APPLICATION_ID=cmohigno7000fun173c4amo69 # Application.id in PMS (pre-created)
PMS_ADMIN_USER_ID=cm1zhu0990000p11oiqpzsn6g  # User.id (john) for LogFile.uploadedBy FK

# App
NEXTAUTH_SECRET=<auto>
```

Implementation Priority

1. **Schema + Seed** — OrderFlow business tables + seed data
 2. **Telemetry Service** — `lib/pms-telemetry.ts` with kill switch
 3. **Basic CRUD UI** — Products, Orders, Customers (functional, minimal styling)
 4. **Telemetry Middleware** — HTTP logger, DB logger, job logger, health emitter
 5. **Chaos Panel** — All 8 scenarios with controls
 6. **Settings Page** — Telemetry toggle + PMS connection status
-

PMS Visibility

Once OrderFlow is running with telemetry enabled, in the PMS app:

- **Dashboard** → filter by “OrderFlow” application → see log volumes, severity distribution
- **System Analysis** → filter by “OrderFlow” → ML anomaly detection on OrderFlow telemetry
- **Predictions** → predictions generated from OrderFlow log patterns
- **Intelligence** → LLM analysis of OrderFlow activity, patterns, risks
- **Data Sources** → OrderFlow appears under its Application (no DataSource needed — direct writes)

The chaos scenarios are designed to trigger specific PMS detection capabilities:

- Error burst → severity escalation detection
- Gradual degradation → trend analysis + early warning
- Cascading failure → cross-source correlation
- Intermittent flapping → pattern recognition